RESILIENCE REPORT

SELF-HEALING ORDER TRACKER

Architecture • Fault Detection • Automated Recovery • Reliability Metrics

PREPARED BY

Adarsh & Abhay

BCA 3rd Year

Project implementation report

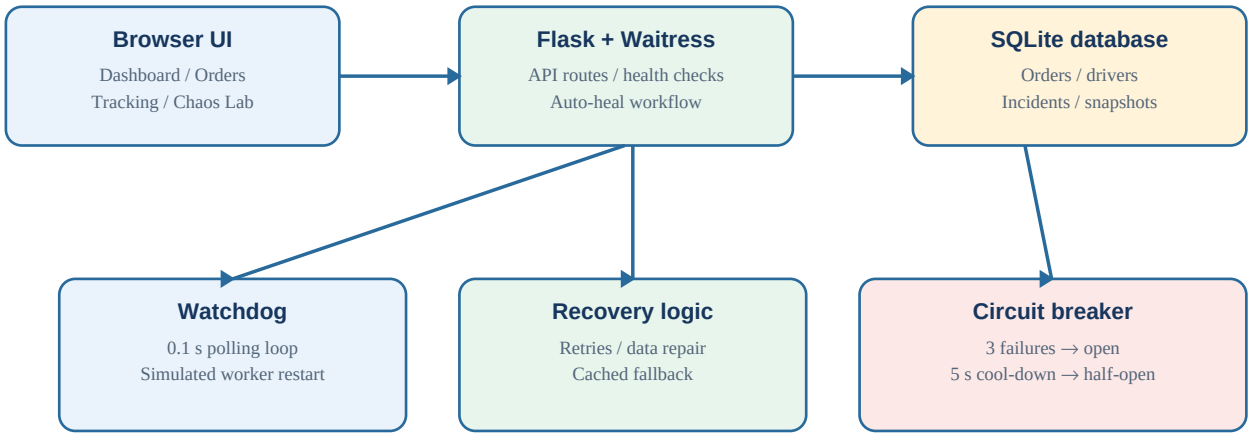
Purpose

This report explains how the application detects selected failures, attempts recovery, records incident stages and calculates reliability metrics. It is based on the supplied project ZIP and README. Configured timings are clearly distinguished from measured test results.

Project stack: Python + Flask/Waitress • SQLite • HTML/CSS/JavaScript • In-process recovery simulations

1. System Architecture

The browser-based manager dashboard and driver portal communicate with a Python Flask application served using Waitress. The backend persists orders, recovery snapshots, incidents and timestamped incident events in SQLite. Runtime health flags and the tracking cache support simulated fault handling.



Request path: browser → API → SQLite/runtime health state. **Recovery path:** fault injection → detection → diagnosis → repair/fallback → verification → incident event saved to SQLite.

Main application components

• **Dashboard:** polls health and component status; “Detect & Auto-Heal” checks supported issues and verifies repairs.
• **Chaos Testing Lab:** injects one of five controlled faults and displays the expected failure beside the observed recovery.
• **Incident timeline:** stores Detected, Diagnosed, Action Taken and Recovered timestamps, mechanism, attempts, detection_ms and recovery_ms.
• **Analytics:** calculates MTTD and MTTR from recovered incident records instead of using fixed demo values.
• **Persistent snapshot:** stores order rows before the clean-up simulation so they can be restored later.

Important boundary

These scenarios are in-process simulations. The watchdog restarts the simulated order worker, not the Flask host process. A real production deployment needs a process supervisor/container policy and independent monitoring for host-level crashes.

2. Failure Detection and Healing

The following failure mechanisms are documented in the supplied project README. The recovery times shown are code-configured timing bounds or conditions, not claimed end-to-end benchmark results. Actual durations should be taken from each incident record.

Failure	How it is detected	How it heals	Recovery-time evidence
Simulated worker crash	Watchdog detects worker_healthy = false; polls about every 0.1 seconds.	Sets simulated worker healthy and records incident stages.	Detection polling interval ≈ 0.1 s; end-to-end time is in incident recovery_ms.
Provider timeout	Tracking/provider call fails; retry attempt is recorded.	Retries after 100 ms and 200 ms before simulated request succeeds.	Configured waits total 300 ms excluding overhead; not a benchmark.
Tracking provider outage	Three failed tracking calls cause circuit state to open.	Rejects calls during cool-down, serves cached status, then attempts half-open recovery.	Cool-down is about 5 s; service recovery depends on availability.
Invalid order status	Data repair scenario writes an invalid status and validates lifecycle values.	Restores the pre-fault order snapshot/status.	Use incident timestamps; no fixed total duration guaranteed.
Network/tracking unavailable	Runtime network/provider flags mark tracking unavailable.	Self-Heal Network or Detect & Auto-Heal restores simulated flags and verifies health.	Four visible stages of about 7.5 s each (~30 s total) when an issue is found.
Recent-orders preview hidden	Health check detects the dashboard preview is unavailable.	Auto-heal restores preview state; full list remains on Order Tracking.	Workflow pacing is about 30 s total when issues are found.
Order rows removed in simulation	Recovery-pending snapshot indicates rows need restoration.	Restores saved rows from persistent SQLite snapshot.	No fixed duration; use injected_at and recovered_at.

Incident lifecycle

Detected → **Diagnosed** → **Action Taken** → **Recovered.** The final stage should be accepted only after the application's verification check passes. A repair being attempted is not, by itself, proof of successful recovery.

3. Before-and-After Reliability Numbers

The source includes incident timestamps and measured-duration fields. However, the supplied ZIP does not include a verified, repeatable chaos-test result set that would justify claiming a specific percentage improvement. The table below reports what can be stated from implementation and what should be populated from test runs.

Metric	Before / fault condition	After / recovered condition	Number / evidence status
Worker health	Simulated worker flag becomes unhealthy.	Watchdog marks worker healthy and records recovery.	Poll interval ≈ 0.1 s; no verified end-to-end median supplied.
Provider retry	Transient request timeout.	Retry after 100 ms then 200 ms.	Configured retry waits = 300 ms total, excluding overhead.
Circuit breaker	Provider calls fail repeatedly.	Opens after 3 failures; half-open attempt after ~5 s.	Threshold = 3 failures; cool-down ≈ 5 s.
Auto-heal workflow	Supported fault remains visible until repair.	Detect → Diagnose → Heal → Verify.	4 stages × ~7.5 s ≈ 30 s when an issue is found; workflow pacing, not measured repair time.
Detection / recovery time	Fault injection timestamp recorded.	Recovered stage saved after verification.	Use detection_ms and recovery_ms from actual runs; no benchmark value supplied.
Success rate / MTTD / MTTR	Fault outcomes not summarized in a test-results file.	Analytics calculates metrics from recovered incidents.	Populate using repeated runs; do not invent percentages.

How to collect genuine before-and-after numbers

1. Run each chaos scenario at least five times under the same conditions. 2. Record the fault injection time, detection time, recovery time and verification outcome. 3. Run one baseline without auto-healing for comparison where safe. 4. Report median and range, plus successful recoveries / total attempts. 5. Keep screenshots or exported incident records as evidence.

Calculations

Detection time = detected_at – injected_at. **Recovery time** = recovered_at – injected_at. **Recovery success rate** = recovered incidents ÷ total injected incidents × 100%. **MTTD** = mean time to detect; **MTTR** = mean time to recover. Report the sample size and test environment beside each number.

4. Known Limits and Conclusion

Known limits

• **Simulated failures:** chaos scenarios change in-app flags/state; they do not shut down a real ISP, tracking provider, database host or Flask process.
• **Process-level crashes:** the watchdog protects only the simulated worker. A service manager or container restart policy is needed for whole-process recovery.
• **Cached tracking:** the last-known order status may be stale while the provider is unavailable. The UI should not interpret cached status as live GPS.
• **Database durability:** SQLite snapshots help recover the deliberate clean-up scenario but are not a replacement for scheduled, tested backups and off-host copies.
• **Security:** driver portal URLs contain bearer tokens; anyone with the URL may act as that driver. Keep links private and use HTTPS. Legacy order-ID location endpoints are unauthenticated and should not be used with real customer data without access controls.
• **GPS constraints:** browser location requires permission, connectivity and HTTPS (or localhost). The app cannot force permission or invent missing coordinates.
• **Performance claims:** timer constants and workflow pacing are not the same as measured recovery time. Results vary with machine load, database I/O and request latency.

Recommended improvements

• Add automated integration tests for each chaos scenario and repeat them in a staging environment.
• Persist/export test results with app version, timestamp, fault type, environment, recovery_ms and success/failure outcome.
• Add external uptime checks and a process supervisor for host-level crashes.
• Strengthen authentication/authorization for location APIs; protect driver tokens and enforce HTTPS.
• Add scheduled backups and periodically test restoring them. Use bounded retries with backoff and jitter for real external dependencies.

Conclusion

The Self-Healing Order Tracker demonstrates five resilience patterns: watchdog restart of a simulated worker, exponential backoff, circuit breaking, cached tracking fallback and snapshot-based data repair. It also records an incident lifecycle and provides analytics for measured recovery. The next step is to run repeatable chaos tests and use those actual records to report median recovery time, recovery success rate, MTTD and MTTR.

Prepared by: Adarsh & Abhay **Department: BCA 3rd Year**